

Система діалогової взаємодії з великими мовними моделями з веб- та консольним інтерфейсами

Зміст

1	АНАЛІТИЧНИЙ ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ	2
1.1.	Характеристика предметної області діалогових систем на основі великих мовних моделей	2
1.2.	Аналіз підходів до побудови програмних систем для взаємодії з LLM . .	2
1.3.	Огляд існуючих аналогів і подібних програмних рішень	3
1.4.	Порівняльний аналіз існуючих рішень	3
1.5.	Недоліки існуючих рішень та обґрунтування необхідності власної розробки	5
1.6.	Постановка задачі розробки програмної системи	5
2	ПРОЄКТУВАННЯ СИСТЕМИ ДІАЛОГОВОЇ ВЗАЄМОДІЇ З LLM	6
2.1.	Вибір архітектури програмної системи	6
2.2.	Формування функціональних і нефункціональних вимог	7
2.3.	Проектування основних функціональних модулів	8
2.4.	Проектування структури бази даних	9
2.5.	UML-діаграма прецедентів	11
2.6.	UML-діаграма класів	12
2.7.	UML-діаграма послідовності	13
2.8.	Опис логічної структури системи	13
	СПИСОК ДЖЕРЕЛ ДЛЯ РОЗДІЛІВ 1–2	14

РОЗДІЛ 1. АНАЛІТИЧНИЙ ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ

1.1. Характеристика предметної області діалогових систем на основі великих мовних моделей

Сучасні програмні системи дедалі частіше використовують засоби штучного інтелекту для автоматизованої обробки текстової інформації, підтримки користувача, генерації відповідей, пояснення навчального матеріалу, написання програмного коду та організації діалогової взаємодії. Одним із найбільш поширених напрямів розвитку таких систем є використання великих мовних моделей, або Large Language Models (LLM).

Великі мовні моделі — це програмні компоненти, побудовані на основі нейронних мереж, які здатні обробляти природну мову, аналізувати контекст запиту та генерувати текстові відповіді. У прикладних системах LLM зазвичай не використовуються ізольовано: навколо моделі створюється програмна інфраструктура, яка забезпечує введення повідомлень користувача, передавання запиту до моделі, отримання відповіді, збереження історії діалогу та відображення результату в зручному інтерфейсі.

Предметна область даної курсової роботи охоплює розробку програмної системи для діалогової взаємодії з великою мовною моделлю. Така система повинна забезпечувати користувачу можливість створювати окремі діалоги, надсилати повідомлення, отримувати відповіді від мовної моделі, переглядати історію листування та працювати з програмою як через веб-інтерфейс, так і через консольний режим.

Особливістю обраної предметної області є поєднання кількох напрямів програмної інженерії:

- побудова користувацького інтерфейсу для діалогової взаємодії;
- організація бізнес-логіки керування чатами та повідомленнями;
- інтеграція pretrained-моделі через окремий LLM-backend;
- збереження історії діалогів у реляційній базі даних PostgreSQL;
- створення модульної архітектури, що дозволяє надалі підключати інші моделі або адаптери;
- забезпечення тестованості основних компонентів системи.

Розроблювана система орієнтована на локальне або навчально-дослідницьке використання. На відміну від багатьох хмарних сервісів, вона передбачає можливість запуску на персональному комп'ютері, використання локально завантажених pretrained-моделей та контроль над структурою програмного продукту.

1.2. Аналіз підходів до побудови програмних систем для взаємодії з LLM

Побудова LLM-системи може виконуватися різними способами. Найпростішим варіантом є використання готового хмарного сервісу, де користувач взаємодіє з моделлю

через браузер або API. Такий підхід мінімізує складність розгортання, але зменшує контроль над моделлю, способом збереження даних і внутрішньою архітектурою.

Інший підхід полягає у використанні локальних інструментів для запуску open-source або open-weight моделей. У цьому випадку модель завантажується на комп'ютер користувача, а генерація відповідей виконується локально. Перевагою такого підходу є більший контроль над даними та можливість експериментувати з різними моделями. Недоліком є залежність від апаратних ресурсів комп'ютера.

Третій підхід — створення власної прикладної системи, яка інтегрує LLM як один із модулів. У цьому випадку мовна модель є не самостійним продуктом, а частиною програмної архітектури. Саме цей підхід обрано в даній курсовій роботі. Він дозволяє відокремити користувацький інтерфейс, бізнес-логіку, сховище даних і LLM-backend.

1.3. Огляд існуючих аналогів і подібних програмних рішень

Для визначення доцільності розробки власної системи було розглянуто декілька існуючих програмних рішень, пов'язаних із діалоговою взаємодією з LLM: ChatGPT, Claude, Google Gemini, LM Studio, Ollama та Open WebUI.

ChatGPT, Claude та Google Gemini є хмарними діалоговими сервісами, які надають користувачу зручний web-інтерфейс і високу якість відповідей. Однак такі системи не дають повного контролю над моделлю, способом збереження історії та внутрішньою архітектурою.

LM Studio та Ollama орієнтовані на локальний запуск моделей. Вони спрощують роботу з pretrained-моделями, але не є навчальними програмними системами, у яких користувач самостійно проєктує web-рівень, CLI-рівень, бізнес-логіку, сховище даних і LLM-backend.

Open WebUI є self-hosted web-інтерфейсом для взаємодії з LLM-backend-ами. Це функціонально близьке рішення, однак воно є готовою платформою, тоді як у даній курсовій роботі важливим є проєктування та реалізація власної модульної системи.

1.4. Порівняльний аналіз існуючих рішень

Порівняльний аналіз аналогів наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння існуючих рішень для діалогової взаємодії з LLM

Рішення	Тип системи	Локальний запуск	Web	CLI	Власна БД	Навчальне розширення
ChatGPT	Хмарний сервіс	Ні	Так	Ні	Ні	Обмежене
Claude	Хмарний сервіс	Ні	Так	Ні	Ні	Обмежене
Google Gemini	Хмарний сервіс	Ні	Так	Ні	Ні	Обмежене
LM Studio	Локальний застосунок	Так	Так	Частково	Обмежено	Обмежене
Ollama	Локальний LLM-runner	Так	Ні / через сторонні UI	Так	Ні	Середнє
Open WebUI	Self-hosted web UI	Через backend	Так	Ні	Так	Середнє
Розроблювана система	Навчальна модульна LLM-система	Так	Так	Так	Так, PostgreSQL	Високе

1.5. Недоліки існуючих рішень та обґрунтування необхідності власної розробки

Аналіз аналогів показує, що існуючі рішення мають низку обмежень у контексті навчальної розробки LLM-системи. Хмарні сервіси не дозволяють повністю контролювати модель, механізм її запуску та структуру збереження діалогів. Локальні інструменти для запуску моделей часто зосереджені саме на інференсі, але не завжди забезпечують навчально корисну архітектуру з чітким поділом на web-рівень, CLI-рівень, бізнес-логіку, storage-рівень і LLM-рівень.

Тому розробка власної системи є обґрунтованою. Вона дозволяє реалізувати програмний продукт, який поєднує web-інтерфейс, консольний режим, збереження історії в PostgreSQL, LLM-backend із підтримкою mock-режиму та локальної pretrained-моделі, а також основу для подальшого розширення через LoRA/QLoRA-адаптацію. У межах поточної реалізації LoRA/QLoRA не використовується і розглядається лише як можливий напрям подальшого розвитку.

1.6. Постановка задачі розробки програмної системи

Метою розробки є створення програмної системи для діалогової взаємодії з великою мовною моделлю з підтримкою web- та консольного інтерфейсів.

Для досягнення поставленої мети необхідно розв'язати такі задачі:

1. Розробити структуру програмного застосунку з головним файлом запуску `app.py`.
2. Реалізувати web-інтерфейс для створення, вибору, перейменування та видалення чатів.
3. Реалізувати консольний режим роботи для взаємодії з чатами через командний рядок.
4. Реалізувати модуль керування чатами та повідомленнями.
5. Організувати збереження історії чатів і повідомлень у PostgreSQL.
6. Реалізувати LLM-backend із можливістю використання mock-моделі та pretrained-моделі через Hugging Face Transformers.
7. Забезпечити підтримку локально завантаженої instruction-моделі.
8. Додати відображення Markdown, блоків коду та математичних формул у web-інтерфейсі.
9. Реалізувати автоматизовані тести для основних компонентів системи.
10. Передбачити можливість подальшого розширення системи, зокрема підтримку LoRA/QLoRA-адаптації.

РОЗДІЛ 2. ПРОЄКТУВАННЯ СИСТЕМИ ДІАЛОГОВОЇ ВЗАЄМОДІЇ З LLM

2.1. Вибір архітектури програмної системи

Для розробки системи діалогової взаємодії з LLM обрано модульну багаторівневу архітектуру. Такий підхід дозволяє розділити відповідальність між окремими компонентами системи та забезпечити можливість подальшого розширення без істотної зміни існуючого коду.

Загальна архітектура системи складається з таких рівнів:

- рівень користувацької взаємодії;
- рівень бізнес-логіки;
- рівень збереження даних;
- рівень взаємодії з мовною моделлю;
- рівень конфігурації та допоміжних сервісів.

Рівень користувацької взаємодії представлений web-інтерфейсом і CLI-режимом. Рівень бізнес-логіки реалізовано у компоненті `ChatManager`. Рівень збереження даних реалізовано через `PostgresStorage`, а рівень взаємодії з мовною моделлю — через `BaseLLMService`, `MockLLMService` і `TransformersLLMService`.

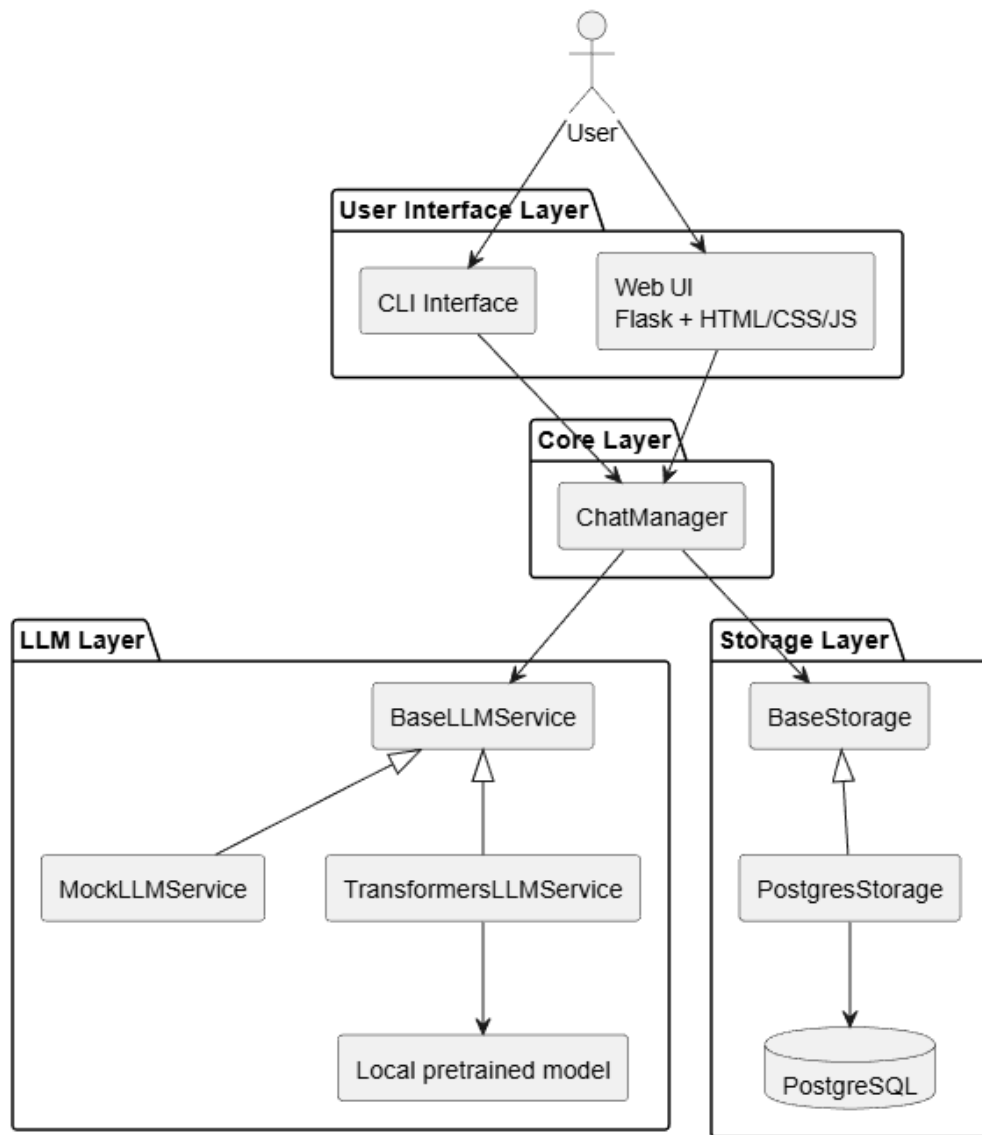


Рисунок 2.1 – Загальна архітектура системи

2.2. Формування функціональних і нефункціональних вимог

На основі технічного завдання та аналізу предметної області сформовано функціональні та нефункціональні вимоги до системи.

Таблиця 2.1 – Функціональні вимоги до системи

Код	Вимога
F1	Система повинна запускатися з головного файла <code>app.py</code> .
F2	Система повинна підтримувати web-режим роботи.
F3	Система повинна підтримувати CLI-режим роботи через параметр <code>-t</code> .
F4	Користувач повинен мати можливість створювати новий чат.
F5	Користувач повинен мати можливість відкривати наявний чат.
F6	Користувач повинен мати можливість перейменовувати чат.
F7	Користувач повинен мати можливість видаляти чат.

Код	Вимога
F8	Користувач повинен мати можливість надсилати повідомлення.
F9	Система повинна генерувати відповідь через підключений LLM-backend.
F10	Система повинна зберігати історію чатів і повідомлень у PostgreSQL.
F11	Система повинна підтримувати mock-режим для тестування.
F12	Система повинна підтримувати Transformers-backend для локальної pretrained-моделі.
F13	Система повинна відображати Markdown, блоки коду та математичні формули у відповідях асистента.
F14	Система повинна показувати активний LLM-backend, назву моделі та preset генерації.

Таблиця 2.2 – Нефункціональні вимоги до системи

Код	Вимога
NF1	Архітектура системи повинна бути модульною.
NF2	Компоненти системи повинні мати слабке зв'язування.
NF3	Система повинна бути придатною до подальшого розширення.
NF4	Основна бізнес-логіка повинна бути покрита автоматизованими тестами.
NF5	Система повинна коректно обробляти помилки користувача.
NF6	Web-інтерфейс повинен бути зручним і зрозумілим.
NF7	Дані чатів повинні зберігатися у структурованому вигляді.
NF8	Конфігурація системи повинна виконуватися через змінні середовища або <code>.env</code> файл.
NF9	Система не повинна жорстко залежати від конкретної мовної моделі.
NF10	Система повинна підтримувати роботу на персональному комп'ютері з Python 3.10 або вище.

2.3. Проектування основних функціональних модулів

Програмну систему поділено на такі основні модулі: web-інтерфейс, CLI-інтерфейс, модуль керування чатами, LLM-backend, модуль збереження даних і конфігураційний модуль.

Модуль web-інтерфейсу відповідає за взаємодію користувача із системою через браузер. CLI-модуль забезпечує альтернативний текстовий режим роботи. Модуль **ChatManager** координує операції між інтерфейсами користувача, сховищем даних і LLM-сервісом. LLM-модуль реалізує генерацію відповідей через mock-режим або локальну pretrained-модель. Модуль збереження даних використовує PostgreSQL для збереження чатів і повідомлень.

2.4. Проектування структури бази даних

Для збереження історії діалогів використовується реляційна база даних PostgreSQL. Структура бази даних складається з двох основних таблиць: `chats` і `messages`.

Таблиця 2.3 – Структура таблиці `chats`

Поле	Тип	Опис
<code>id</code>	TEXT	Унікальний ідентифікатор чату
<code>title</code>	TEXT	Назва чату
<code>created_at</code>	TIMESTAMPTZ	Дата і час створення чату
<code>updated_at</code>	TIMESTAMPTZ	Дата і час останнього оновлення чату

Таблиця 2.4 – Структура таблиці `messages`

Поле	Тип	Опис
<code>id</code>	BIGSERIAL	Унікальний ідентифікатор повідомлення
<code>chat_id</code>	TEXT	Ідентифікатор чату, до якого належить повідомлення
<code>role</code>	TEXT	Роль автора повідомлення: <code>user</code> або <code>assistant</code>
<code>content</code>	TEXT	Текст повідомлення
<code>timestamp</code>	TIMESTAMPTZ	Дата і час створення повідомлення
<code>position</code>	INTEGER	Порядковий номер повідомлення в чаті

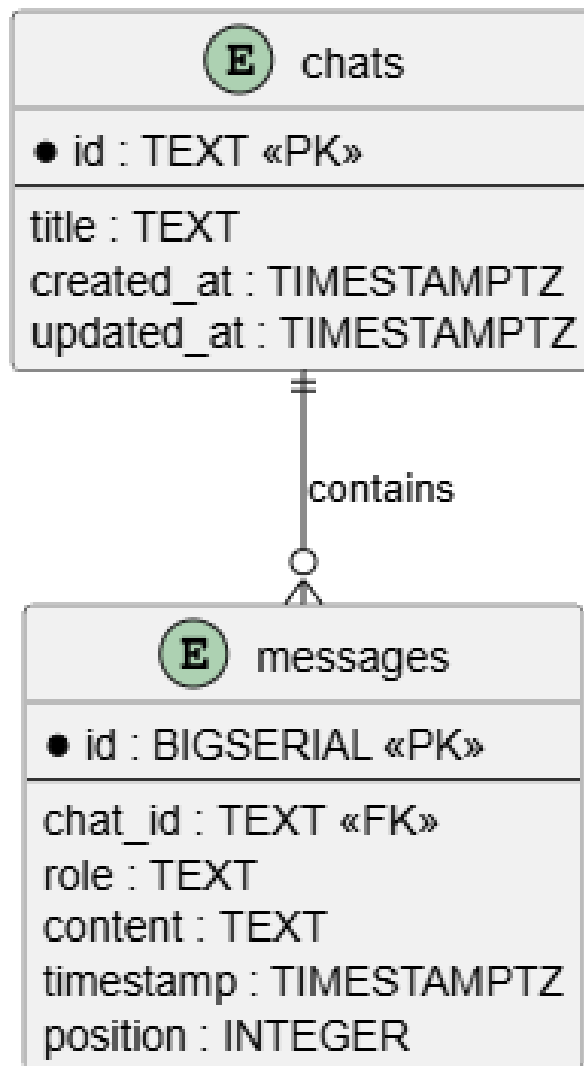


Рисунок 2.2 – ER-діаграма бази даних

2.5. UML-діаграма прецедентів

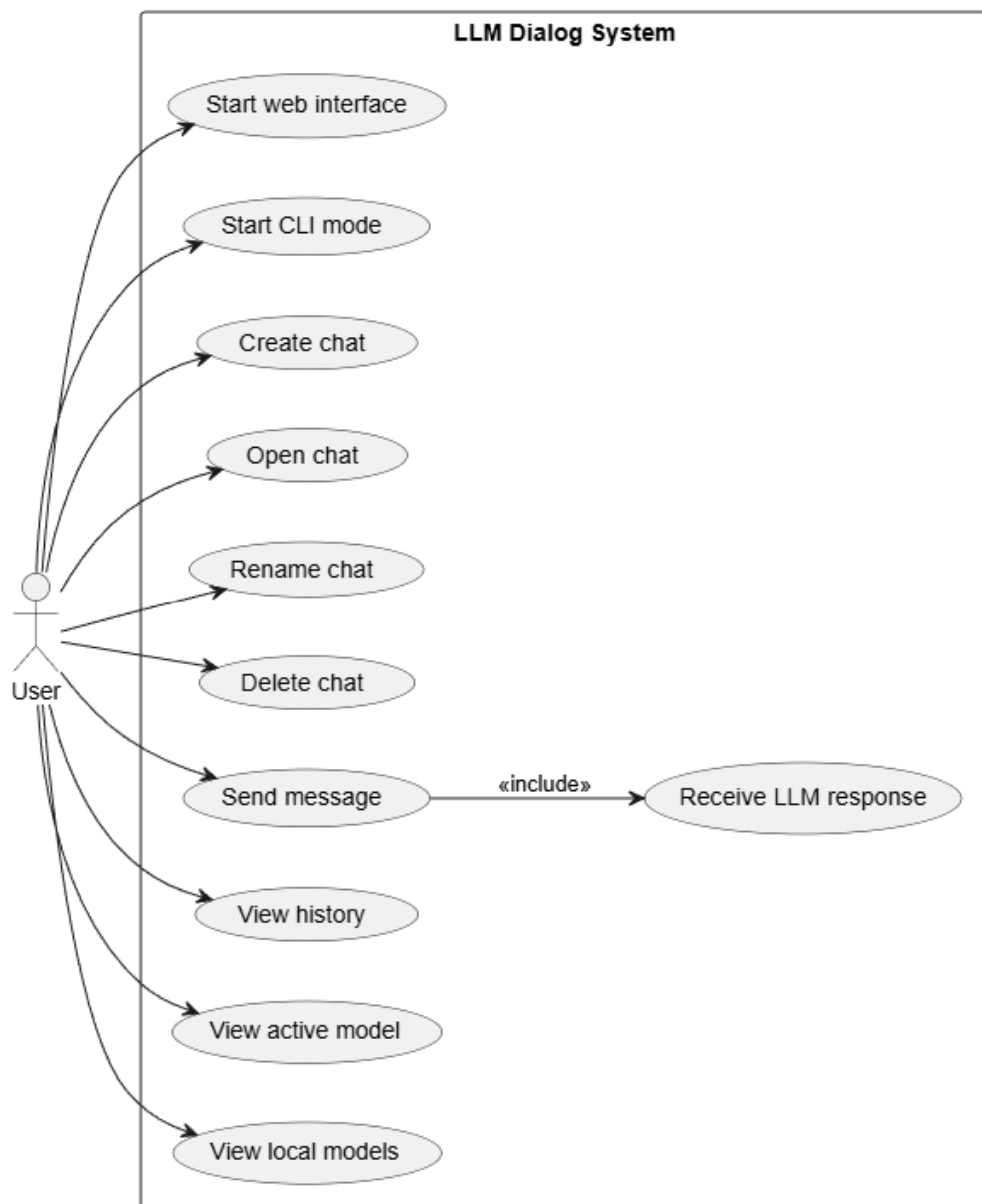


Рисунок 2.3 – UML-діаграма прецедентів системи

2.6. UML-діаграма класів

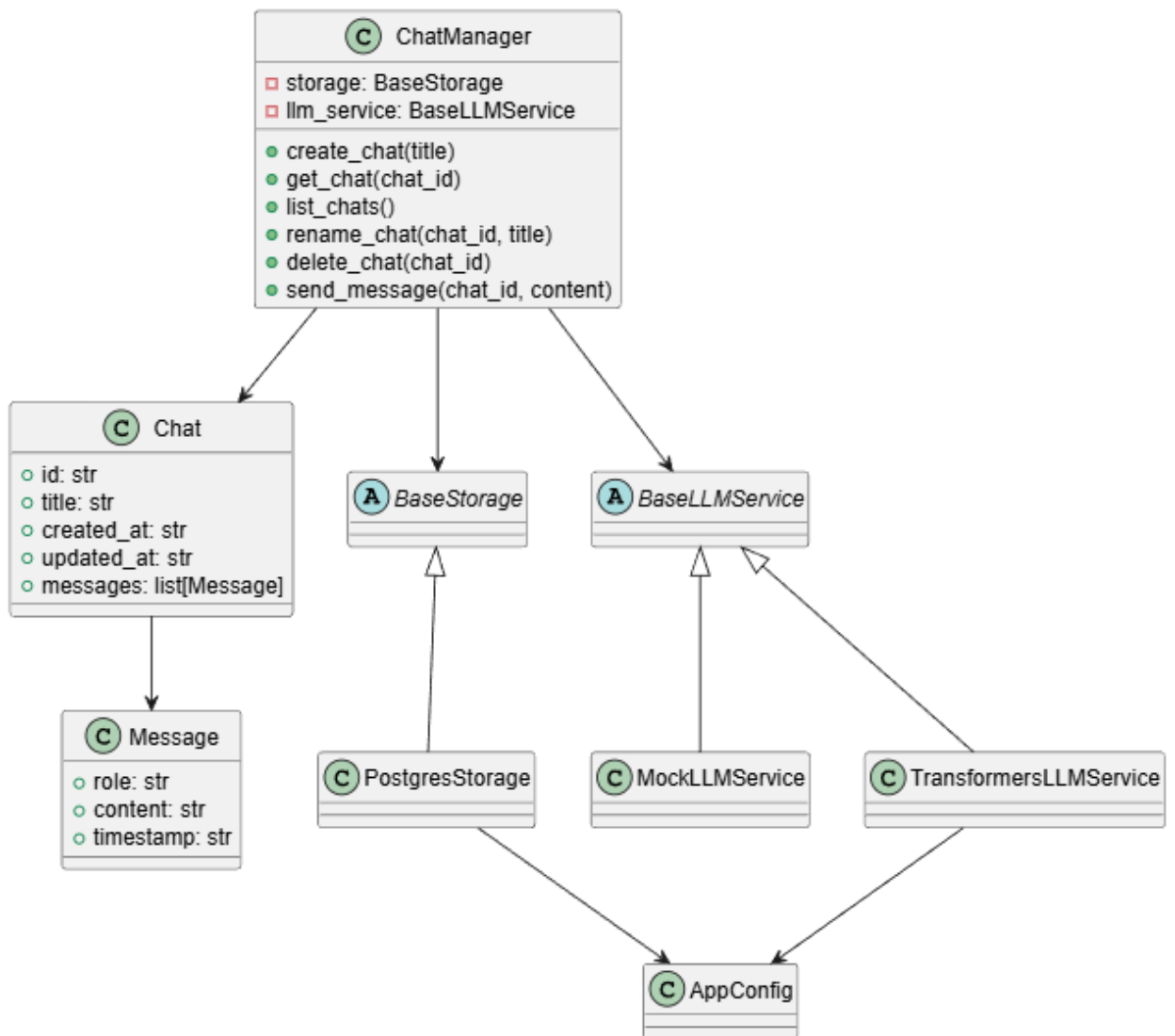


Рисунок 2.4 – UML-діаграма класів системи

2.7. UML-діаграма послідовності

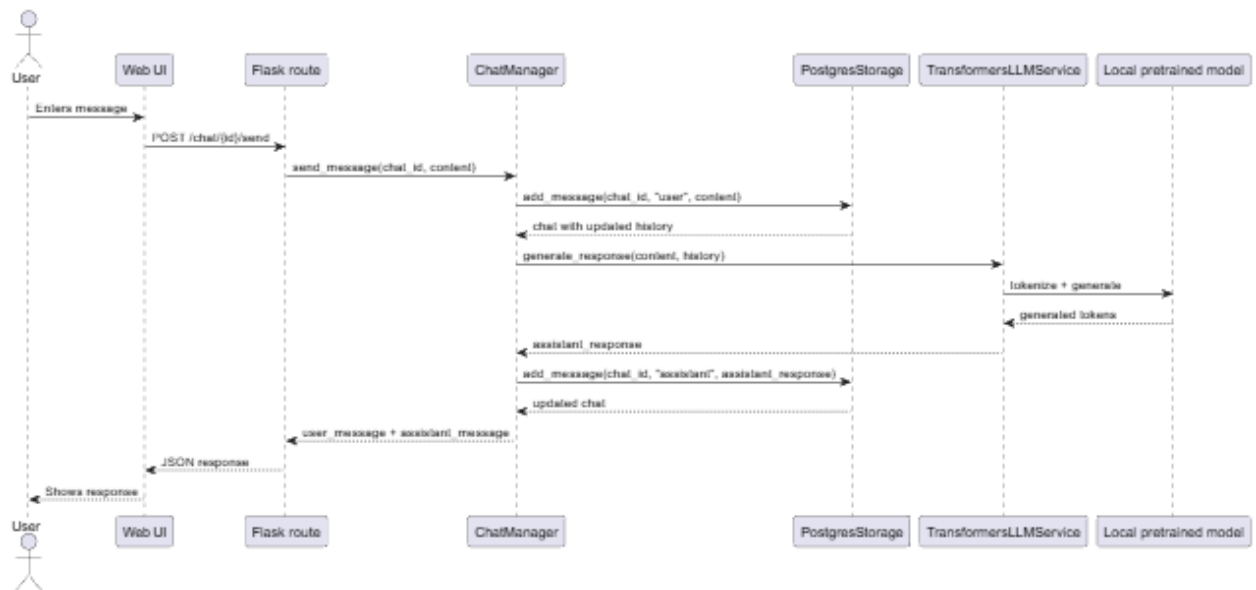


Рисунок 2.5 – UML-діаграма послідовності надсилання повідомлення

2.8. Опис логічної структури системи

Логічна структура системи побудована так, щоб окремі частини програмного продукту мали чітко визначені обов'язки. Web-інтерфейс і CLI-інтерфейс не працюють безпосередньо з базою даних або моделлю. Вони звертаються до **ChatManager**, який координує виконання операцій.

ChatManager не залежить від конкретної реалізації LLM-рівня. Для генерації відповідей використовується інтерфейс **BaseLLMService**. Для збереження даних у поточній реалізації використовується **PostgresStorage**, що працює з PostgreSQL.

PostgreSQL використовується як основне сховище даних. Це дозволяє зберігати історію діалогів у структурованому вигляді, підтримувати зв'язок між чатами й повідомленнями та надалі розширювати схему бази даних.

LLM-рівень підтримує два режими: mock-режим і Transformers-режим. Mock-режим використовується для тестування та швидкого запуску без локальної моделі. Transformers-режим дозволяє підключати pretrained-модель, завантажену в локальну директорію `models/`. На поточному етапі система не реалізує потокову генерацію токенів і не підтримує перемикання моделей під час виконання без перезапуску застосунку.

Конфігурація системи виконується через змінні середовища або `.env` файл. Це дозволяє змінювати LLM-backend, модель, preset генерації та параметри підключення до PostgreSQL без зміни вихідного коду.

Таким чином, обрана структура забезпечує модульність, розширюваність, зручність тестування та відповідність поставленим вимогам до курсового проєкту.

СПИСОК ДЖЕРЕЛ ДЛЯ РОЗДІЛІВ 1–2

1. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser L., Polosukhin I. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017. URL: <https://arxiv.org/abs/1706.03762>.
2. Zhao W. X., Zhou K., Li J., Tang T., Wang X., Hou Y. та ін. A Survey of Large Language Models. arXiv preprint, 2023. URL: <https://arxiv.org/abs/2303.18223>.
3. Wolf T., Debut L., Sanh V., Chaumond J., Delangue C., Moi A. та ін. Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP: System Demonstrations, 2020. URL: <https://aclanthology.org/2020.emnlp-demos.6/>.
4. Hugging Face. Transformers documentation. URL: <https://huggingface.co/docs/transformers/index>.
5. Hugging Face. Chat templates documentation. URL: https://huggingface.co/docs/transformers/en/chat_templating.
6. Hugging Face. Text generation documentation. URL: https://huggingface.co/docs/transformers/en/llm_tutorial.
7. Qwen Team. Qwen2.5 Technical Report. arXiv preprint, 2024. URL: <https://arxiv.org/abs/2412.15115>.
8. Hugging Face. Qwen2.5-0.5B-Instruct model card. URL: <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>.
9. Flask. Flask Documentation. URL: <https://flask.palletsprojects.com/>.
10. Flask. Quickstart. URL: <https://flask.palletsprojects.com/en/stable/quickstart/>.
11. PostgreSQL. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/>.
12. PostgreSQL. Tutorial. URL: <https://www.postgresql.org/docs/current/tutorial.html>.
13. PostgreSQL. Database Concepts. URL: <https://www.postgresql.org/docs/current/tutorial-concepts.html>.
14. Python Software Foundation. Python Documentation. URL: <https://docs.python.org/3/>.
15. PlantUML. PlantUML Documentation. URL: <https://plantuml.com/>.
16. LM Studio. Documentation. URL: <https://lmstudio.ai/docs>.
17. Ollama. Documentation. URL: <https://github.com/ollama/ollama/tree/main/docs>.
18. Open WebUI. Documentation. URL: <https://docs.openwebui.com/>.